

Web 应用为主体

Sak-AI答题助手

面向备考学习与题库运营场景的单仓全栈 Web 应用

题库广场 | 个人题库 | 刷题练习 | 模拟考试 | 数据中心 | 论坛互动 | 后台管理

不是单一刷题页面，而是围绕“题库沉淀—训练—复盘—运营”构建的完整业务系统。

12+

业务模块

Blueprint 按边界拆分

2 端

共享语义

Web + 微信原生小程序

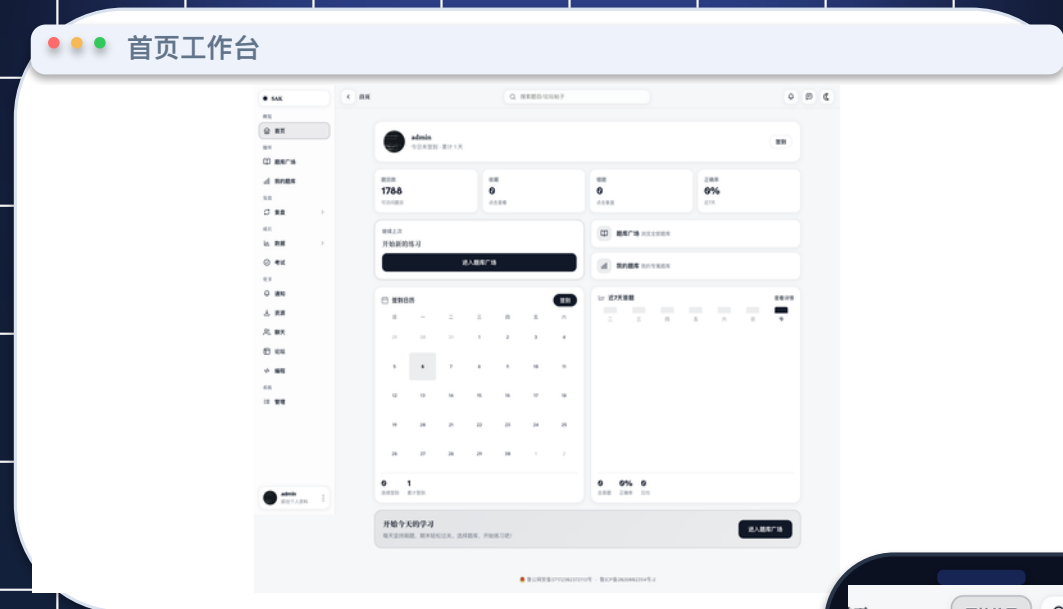
Docker

可复现部署

web / worker /
postgres / redis

赛道：软件应用与开发 | Web 应用
与开发

正式访问：<https://saksk.top>



问题背景与需求洞察

从单点刷题工具走向完整学习闭环，是本作品的切入点。

1 资源分散

公共题库、个人题库、课程练习常常分散存在，难以沉淀为持续可复用的学习资产。

2 训练断层

用户能刷题，但难以把练习、考试、错题复盘和阶段统计串成一条连续的成长链路。

3 管理割裂

教师或运营侧需要统一的内容管理、数据反馈与社区支撑，传统工具往往只覆盖学习端。

Sak-AI 的回答

以 Web 应用为核心入口，承载完整业务流程。

- 统一承接题库广场、个人题库、刷题、考试、数据中心、论坛与后台。
- 通过单仓全栈架构，让学习端与运营端共用一套后端与数据语义。
- 以微信原生小程序补足移动高频场景，而不是另起一套割裂系统。
- 以 AI 题目解析增强讲解与复盘体验，提升持续学习效率。

Web

主体交互端

SSR + 原生 JS

Mini

移动延展端

微信原生 TS

AI

辅助增强

题目解析与学习说明

作品定位：一套系统，服务学习者与管理者

Web 为主、小程序为辅，共享同一后端语义与学习数据。

学习端

题库浏览与沉淀

公共题库 + 个人题库双线并行，支持加入、整理与长期复用。

练习与考试闭环

错题、收藏、用户答案、考试记录与学习进度持续沉淀。

可视化复盘

数据中心用趋势图、热力图与能力画像，让反馈可见。

共享语义核心

01

题库资源
公共题库 / 个人题库

02

刷题训练
答案、收藏、错题、进度

03

考试评估
模拟考试与结果沉淀

04

数据复盘
趋势、热力图、能力画像

运营端

论坛互动

支持帖子浏览、交流讨论与学习社区沉淀。

后台管理

题目、科目、用户与系统数据可统一查看与管理。

移动延展

微信小程序基于同一后端语义，承接随时随地学习场景。

系统架构：单仓全栈、双端共享、容器化可复现

不是前后端分离多仓拼装，而是一套围绕业务模块组织的工程系统。

1 应用层：Web + 微信原生小程序

Web 采用 Flask + Jinja 服务端渲染，小程序采用微信原生 TS，两个端共享同一后端语义。

2 服务层：Flask 应用工厂 + Blueprint 模块化

已注册 auth、main、quiz、exam、user、chat、notifications、popups、coding、user_bank、forum、admin 等 12 个模块。

3 数据层：PostgreSQL + Redis + RQ

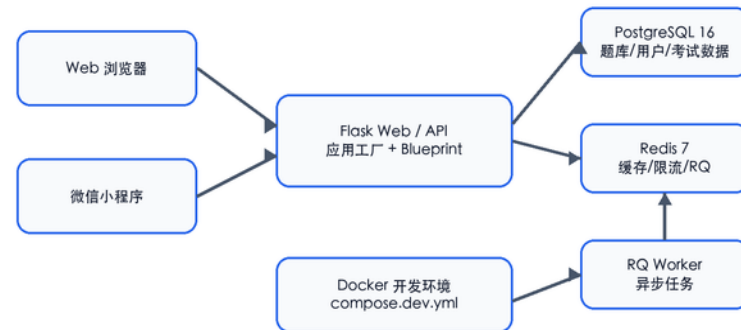
PostgreSQL 存储题库与学习数据，Redis 承担缓存/限流/队列，RQ Worker 处理后台任务。

4 交付层：Docker Compose 开发编排

compose.dev.yml 统一拉起 web、worker、postgres、redis、backup，便于演示、复现与现场切换。

系统架构图

Sak-AI答题助手系统架构图



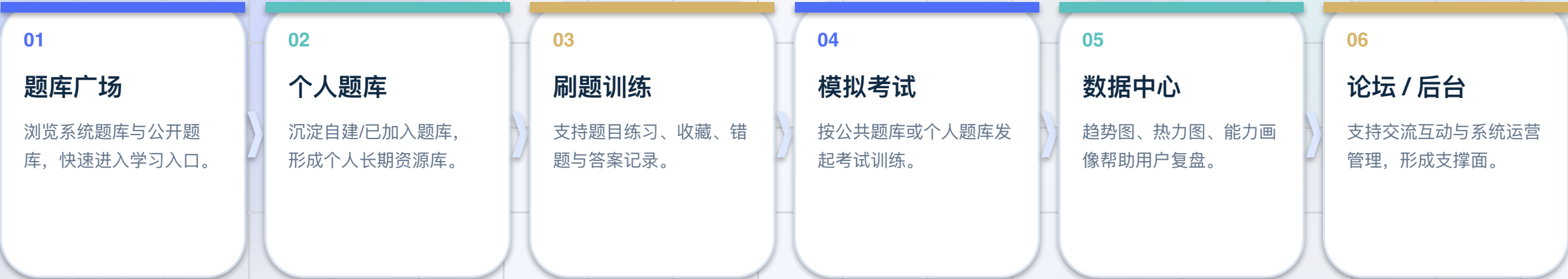
Web 为主应用端，小程序复用同一后端与数据语义。

核心用户流程



核心功能闭环：从题库沉淀到数据复盘

围绕学习者与运营者共同构建完整业务流程。



闭环价值：让“资源组织 → 高频训练 → 阶段评估 → 可视化复盘 → 社区互动 / 运营支撑”在同一系统中连续发生。

真实界面展示：学习入口与题库资产

以下截图均来自作品真实运行界面。



刷题、AI 解析与 AI 判分

重点展示答题工作台、即时解析与主观题判分配置。



主观题 AI 判分设置



模块亮点

- 支持题目列表、答题卡片与解析联动。
- AI 解析可按题触发，辅助理解知识点。
- 主观题支持有答即对 / AI 判分 / 自评模式。

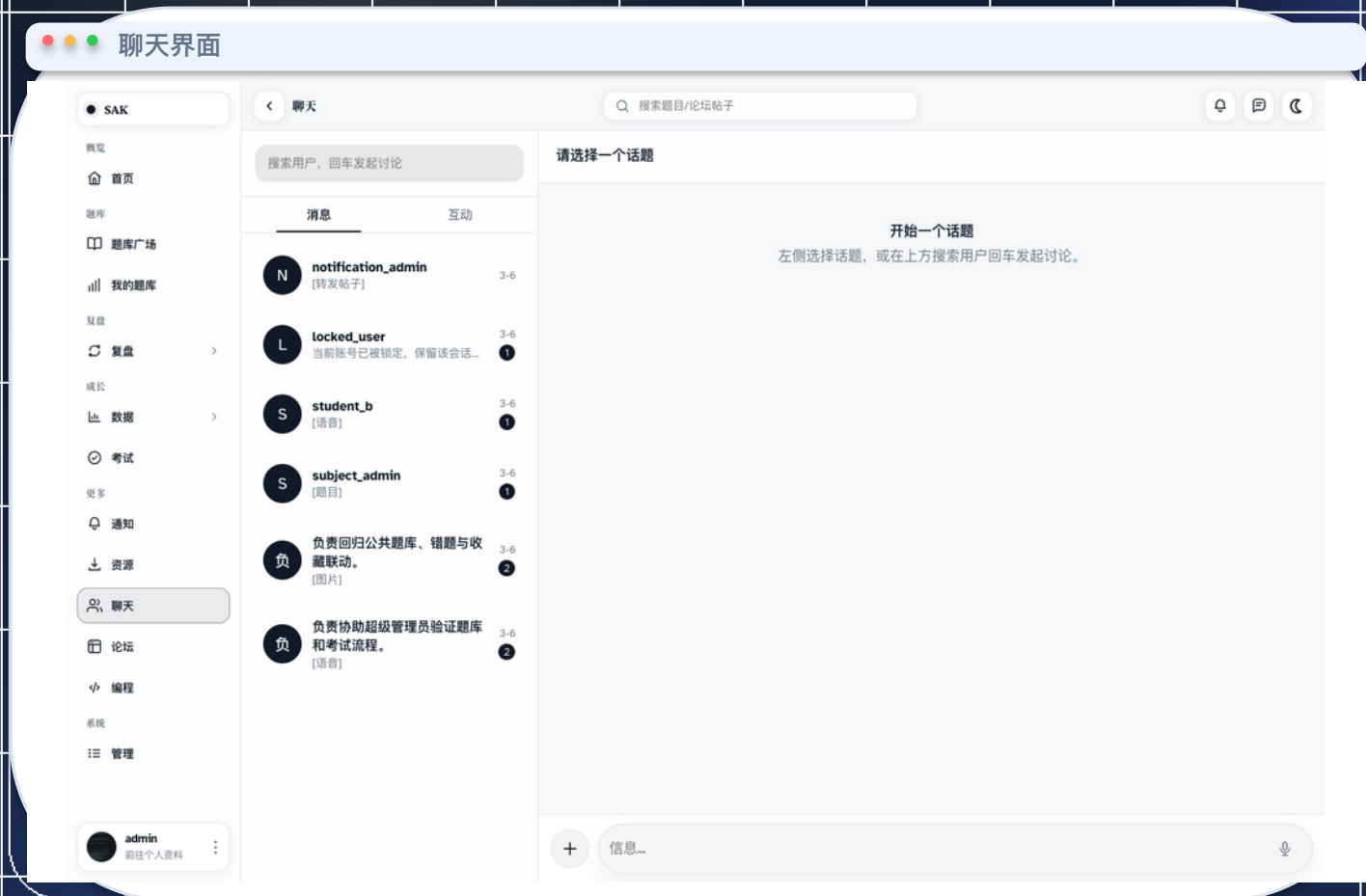
论坛模块：社区首页到帖子详情

不仅有帖子列表，也有完整的帖子阅读、目录、标签与互动结构。



聊天模块：会话、搜索与互动内容承接

支持从资料页或系统入口进入聊天，会话中可承接题目、图片、语音与帖子转发。



交互能力

搜索用户后可直接发起讨论。

- 会话列表区分消息与互动。
- 消息类型覆盖图片、语音、题目与帖子转发。
- 与论坛、个人资料等模块形成联动。

IM

消息流

会话 / 搜索 / 发送

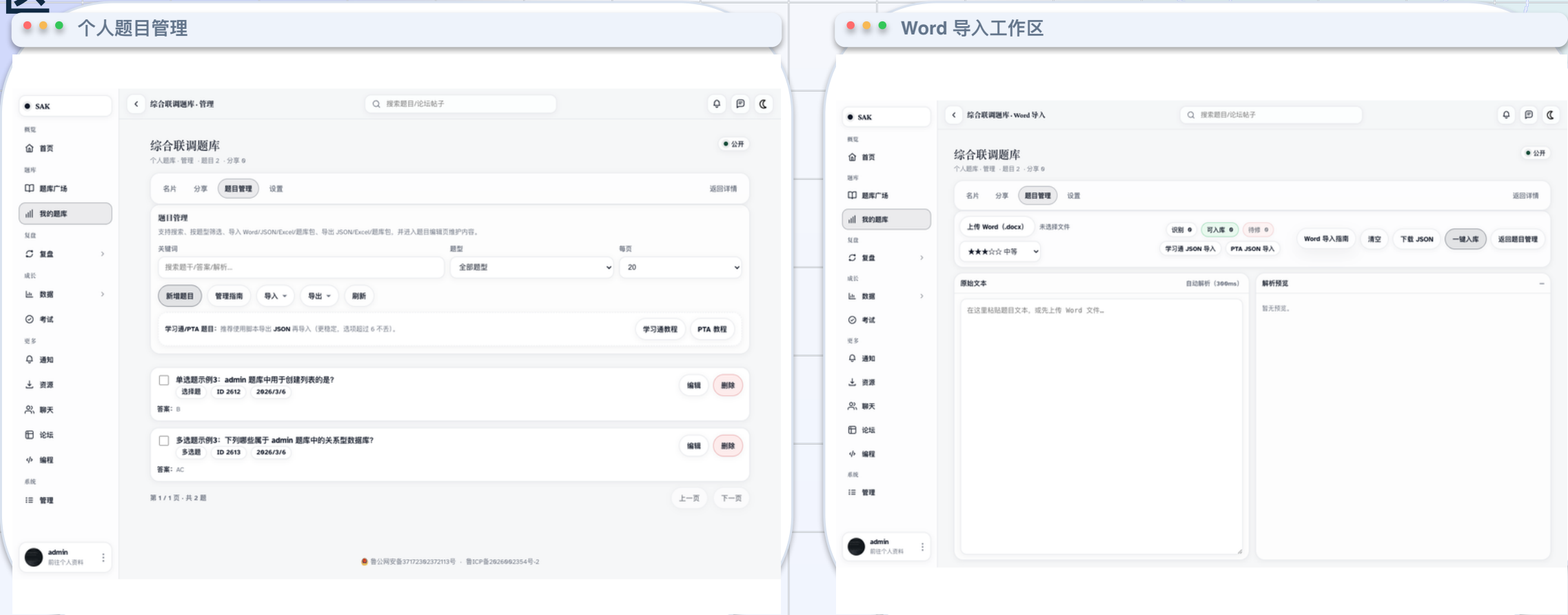
Link

内容互通

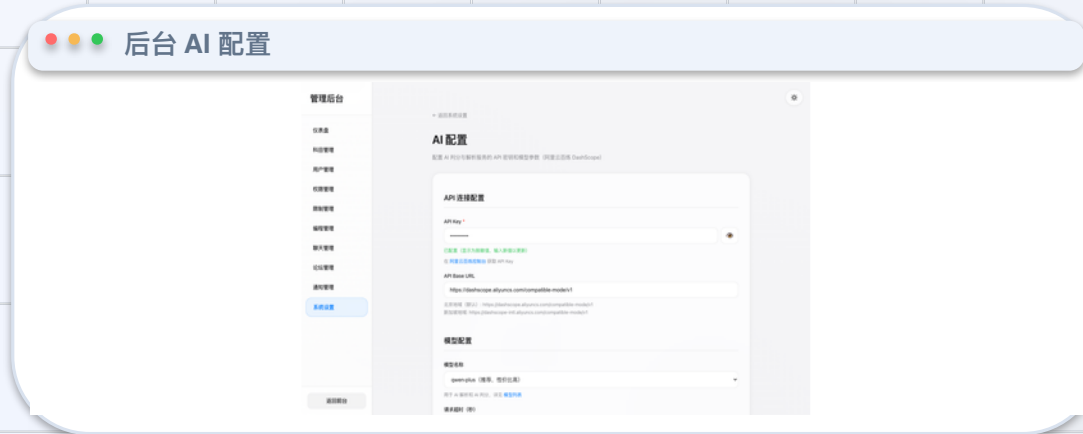
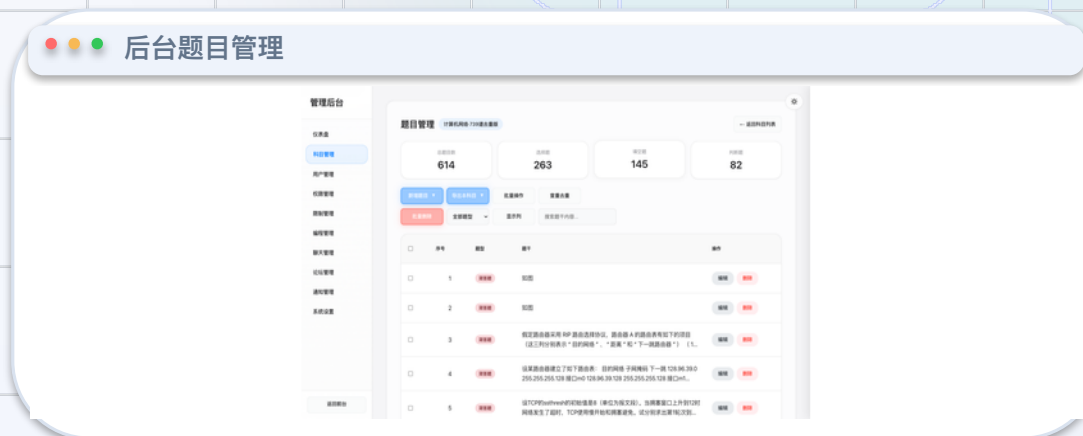
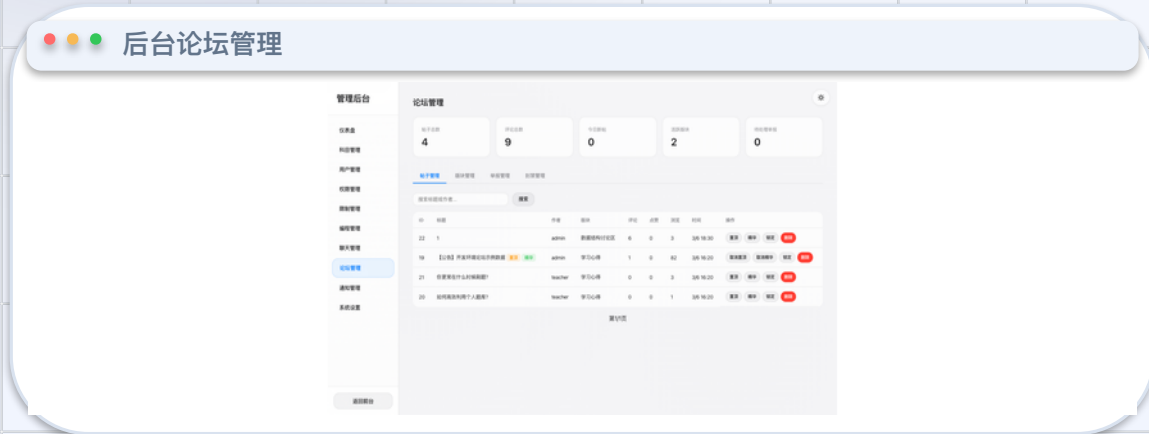
题目 / 帖子 / 图片

个人题目导入：题目管理 + Word 工作

个人题库不只是存储入口，还提供完整的题目导入与维护流程。



后台不仅有仪表盘，还覆盖题目治理、论坛管理与系统级 AI 配置。



双端协同：Web 主体 + 小程序延展

展示主体是 Web；移动端延展同样复用同一套后端与数据语义。



共享关键语义

收藏 / 错题 / 用户答案 / 用户进度 / 模拟考试

- Web 端承担主要交互与展示，是参赛主体。
- 小程序按高频移动学习场景延展，减少语义割裂。
- 统一经过后端与接口封装，降低维护与演示成本。

SSR

Web 形态

Flask + Jinja

JWT

API / 小程序

Bearer Token

关键点：工程化表达与应用价值并重

在题库系统常见能力之外，重点强化共享语义、可复现交付与学习辅助增强。

1 共享数据语义

公共题库、个人题库、题目、收藏、错题、答案、进度、考试等能力在 Web 与小程序两端保持同一套后端语义，降低维护复杂度，也让学习记录真正可连续。

2 Docker 化可复现

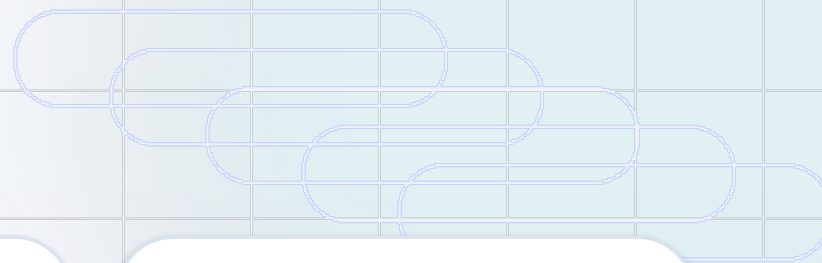
开发环境默认通过 `compose.dev.yml` 统一拉起 `web`、`worker`、`postgres`、`redis`、`backup`。无论课堂演示、现场答辩还是后续交付，都能更稳定地复现运行环境。

3 AI 学习辅助增强

围绕刷题流程补充 AI 解析、主观题判分与后台 AI 配置能力，让系统更贴近教学、练习与复盘的真实场景。

工程验证与答辩可演示性

答辩现场不仅要讲清功能，还要展示项目具备稳定运行与工程交付能力。



8000

Web 暴露端口

compose.dev.yml: flask run --
reload

/api/ping

健康检查

支持基础与 deep=1 检查

PostgreSQL

开发默认数据库

Redis + RQ 协同运行

5 项服务

容器编排

web / worker / postgres / redis / backup

推荐演示顺序

首页 → 题库广场 → 题库详情 → 刷题/AI → 论坛帖子
→ 聊天 → 个人导入 → 后台管理

正式访问地址: <https://saksk.top>

本地演示备用: <http://127.0.0.1:8000>

工程事实

- Web 是主体交互端，采用 Flask + Jinja SSR。
- 小程序为微信原生工程，作为移动场景延展。
- 应用工厂 + Blueprint 模块化，便于后续扩展。
- 新接口遵循 status / code / data / message 信封结构。

SUMMARY

Sak-AI答题助手

让题库沉淀、刷题训练、模拟考试、数据复盘、论坛互动
与平台运营在一个系统中完整闭环。

谢谢各位老师指导

正式访问地址

<https://saksk.top>

现场网络受限时可切换本地 Docker 演示环境。

